15/11/2025

TERM PAPER

Anushka: 202301224
Yashasvi: 202301069

# Integrating Vertical and Horizontal Partitioning into Automated Physical Database Design

**Sanjay Agrawal**
Microsoft Research
sagrawal@microsoft.com

**Vivek Narasayya**
Microsoft Research
viveknar@microsoft.com

**Beverly Yang**
Stanford University
byang@stanford.edu

# Objective & Motivation

- Integrate horizontal and vertical partitioning with indexes and materialized views to automate physical database design.
- Selecting optimal indexes and partitioning strategies are not independent problems.
- The best choice for an index might depend on how the table is partitioned, and likewise, the best partitioning scheme often depends on which indexes are available.
- Hence, we need to check different combinations of possible physical designs.
- But checking every single possible candidate would be computationally difficult (almost impossible).

**Workload**

A set of SQL DML statements. This includes SELECT, INSERT, DELETE and UPDATE statements.
Each statement Q in the workload can optionally be given a weight, f_Q, which represent how frequently it runs.

**CG-Cost(g)**

the fraction of the total cost of all queries in the workload where that column-group g is referenced.

$$CG\text{-}Cost(g) = \frac{\text{Total cost of queries referencing } g}{\text{Total cost of all queries in workload}}$$

**Query Cost**

the optimizer-estimated cost of running a query Q with a given physical design P, represented as Cost(Q, P)

**Alignment**

It mandates that indexes on a table must be horizon tally partitioned in the exact same way as the table itself. This is a key manageability requirement.

**The Physical Design Problem**

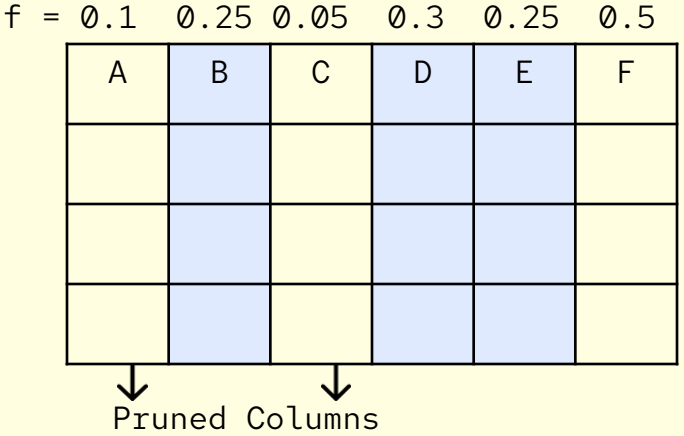
The goal is to find a configuration (a valid set of physical design structures) that minimizes the total optimizer-estimated cost for a given workload W, subject to a total storage budget S. This problem, and its sub-problems (like index selection and partitioning), are **NP-hard.**

Proposed Technique

**1. Column-Group Restriction**

A column-group g is "interesting" if its CG-Cost(g) is above a threshold f.

**Example:** If the threshold f = 0.2 (20%), and queries using column (A) only account for 10% of the total cost, the group {A} is pruned. Any larger group containing A, like {A, D), is also removed.




**2. Candidate Selection**

The optimizer estimates the Cost(Q, P) of different combinations.
The best combinations for each query are selected as "candidates".
Then find the cheapest configuration P for a given query Q.

**3. Merging**

Take the "specialized" candidates (which are good for single queries) and try to merge them to get a generalized solution.
For example, it might merge two different indexes or two different vertical partition schemes.

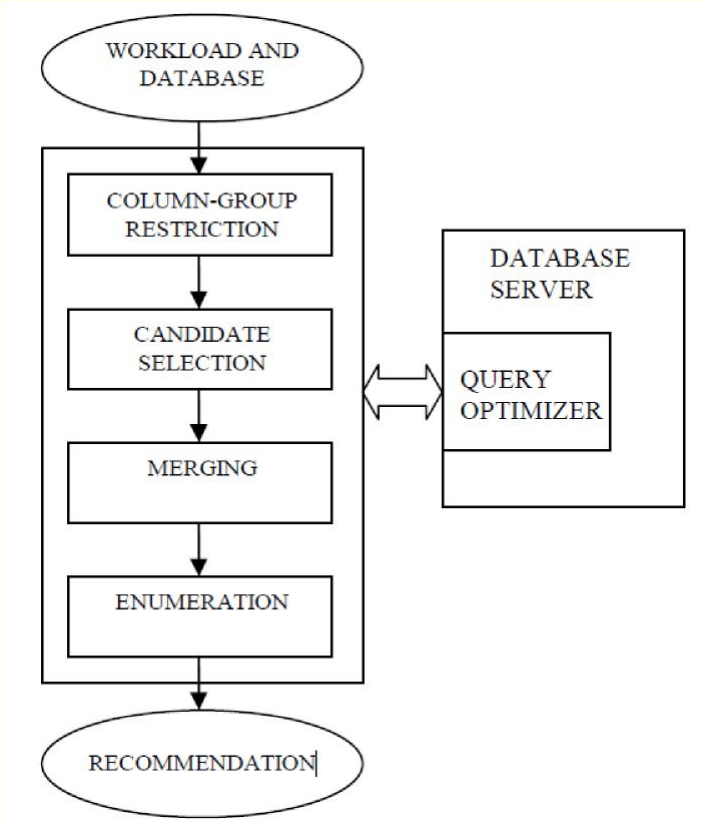
Proposed Technique

### 4. Enumeration

Take all candidates (from selection and merging) as input.

Searches for the best overall subset of these structures for the entire workload.

The goal is to find a configuration P that minimizes the total weighted workload cost.
This is subject to the constraint that the total Storage(P) <= S(the storage bound).



# Experimental Setup and Workloads

**Goal**

Validate the integrated approach (TOGETHER) against staged and index-only methods using real and synthetic workloads.

**Environment**

**Database:** TPC-H 1GB (TPCH1G), APB 1.2 GB
**Hardware:** 1 GHz CPU | 256 MB RAM | Commercial DB Server

**Workloads**

**TPCH22:** 22 standard TPC-H queries (for performance comparison)
**CS-WKLD:** 100 random SPJ queries (for scalability)
**WKLD-COL-n:** 25 queries (20–80% joins) → tests co-location impact
**APB:** 200 complex decision-support queries (for alignment evaluation)

**Evaluation Focus:**

Integrated vs. Staged performanceEffectiveness of Column-Group Restriction
Impact of Co-location in MergingScalability of Lazy Enumeration

# Results

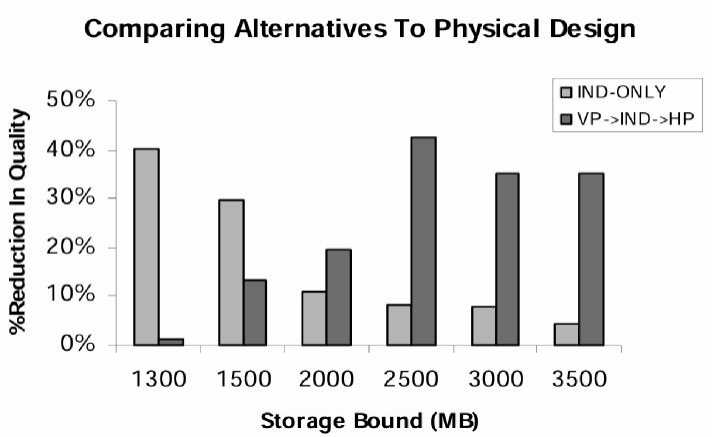



**fig. (a)** Evaluated the solution quality (based on optimizer-estimated cost) of the integrated TOGETHER approach against the IND-ONLY and staged VP→IND→HP methodologies, varying the available storage bound.

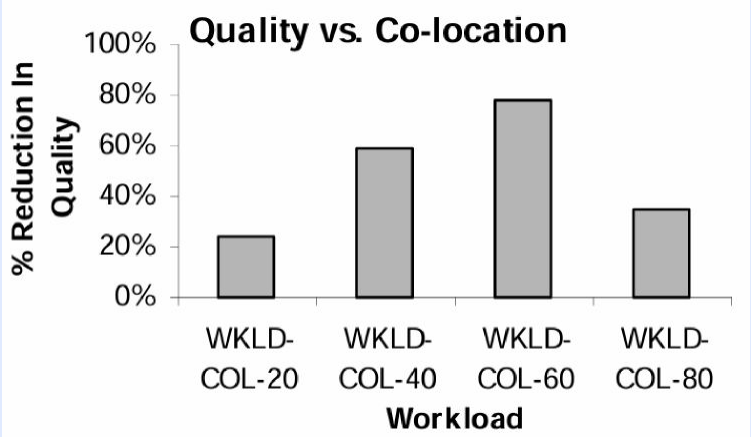



**fig. (b)** Comparison between proposed algorithm for Merging (considering co-location) with a variant of this algorithm (NOCOL) that does not take co-location into consideration.

# Results

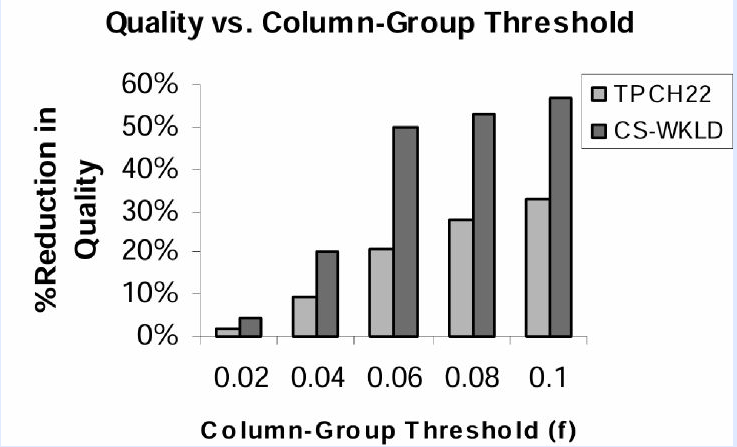



**fig. (c)** The reduction in quality for different values of f for the two workloads

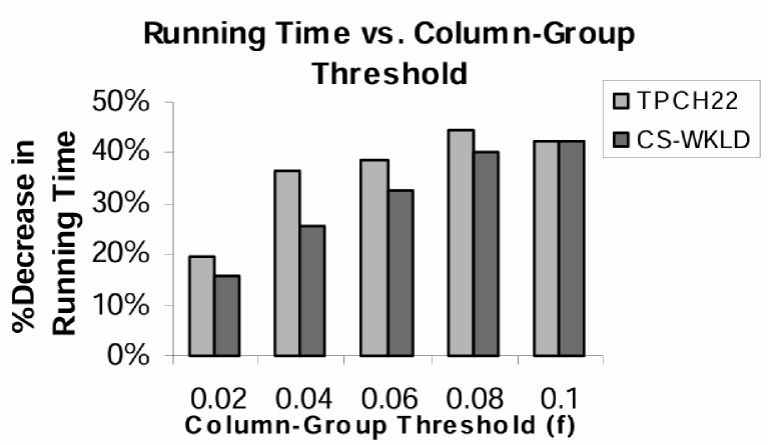



**fig. (d)** The trade-off between solution quality and tool runtime when applying the “Column-Group Restriction” pruning heuristic, controlled by the threshold f

# Future Research Directions

## Expanding Scope and Context

**Multi-Node Partitioning:** Extend the framework to shared-nothing clusters (distributed DBs).Must account for availability and network latency as new cost factors.
**Dynamic & Adaptive Design:** Move beyond static workloads.Develop online systems that monitor and adapt partitioning in real time.

## Modernizing Cost Models and Pruning

**Cloud-Native Cost Models:** Redefine cost estimation for cloud platforms (e.g., Snowflake, BigQuery).Base costs on bytes scanned and compute-seconds, not I/O.
**Machine Learning for Pruning:** Replace heuristic metrics (CG-Cost, VPC) with ML models trained on query traces.Enables smarter, data-driven pruning of design candidates.

# References

Agrawal, S., Narasayya, V., & Yang, B. (2004). Integrating Vertical and Horizontal Partitioning into Automated Physical Database Design. Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data (SIGMOD '04), 359–370.